C++ Conditions and If Statements

You already know that C++ supports the usual logical conditions from mathematics:

- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`
- Equal to `a == b`
- Not Equal to: `a != b`

You can use these conditions to perform different actions for different decisions.

C++ has the following conditional statements:

- Use **`if`** to specify a block of code to be executed, if a specified condition is true
- Use **`else`** to specify a block of code to be executed, if the same condition is false
- Use **`else if`** to specify a new condition to test, if the first condition is false
- Use **`switch`** to specify many alternative blocks of code to be executed

The if Statement

Use the **`if`** statement to specify a block of C++ code to be executed if a condition is **`true`**.

Syntax

```
if (condition) {
  // block of code to be executed if the condition is true
}
```

Note that **if** is in lowercase letters. Uppercase letters (If or IF) will generate an error.

In the example below, we test two values to find out if 20 is greater than 18. If the condition is **true**, print some text:

Example

```
if (20 > 18) {
  cout << "20 is greater than 18";
}
```

```
#include <iostream>
using namespace std;
int main() {
  if (20 > 18) {
    cout << "20 is greater than 18";
  }
  return 0;
}
```

OR

```
#include <iostream>
using namespace std;
int main() {
  int x = 20;
  int y = 18;
  if (x > y) {
    cout << "x is greater than y";
  }
  return 0;
}
```

C++ Else

The else Statement

Use the **`else`** statement to specify a block of code to be executed if the condition is **`false`**.

Syntax

```
if (condition) {
  // block of code to be executed if the condition is true
} else {
  // block of code to be executed if the condition is false
}
```

Example

```
int time = 20;
if (time < 18) {
  cout << "Good day.";
} else {
  cout << "Good evening.";
}
// Outputs "Good evening."
```

C++ Else If

The else if Statement

Use the **`else if`** statement to specify a new condition if the first condition is **`false`**.

Syntax

```
if (condition1) {
  // block of code to be executed if condition1 is true
} else if (condition2) {
  // block of code to be executed if the condition1 is false and
condition2 is true
} else {
  // block of code to be executed if the condition1 is false and
```

condition2 is false}

Example

```
int time = 22;
if (time < 10) {
  cout << "Good morning.";
} else if (time < 20) {
  cout << "Good day.";
} else {
  cout << "Good evening.";
}
// Outputs "Good evening."
```

C++ Switch

C++ Switch Statements

Use the **switch** statement to select one of many code blocks to be executed.

Syntax

```
switch(expression) {
  case n1:
    // code block
    break;
  case n2:
    // code block
    break;
  default:
    // code block
}
```

This is how it works:

- The **switch** expression is evaluated once
- The value of the expression is compared with the values of each **case**
- If there is a match, the associated block of code is executed
- The **break** and **default** keywords are optional, and will be described later in our lecture.

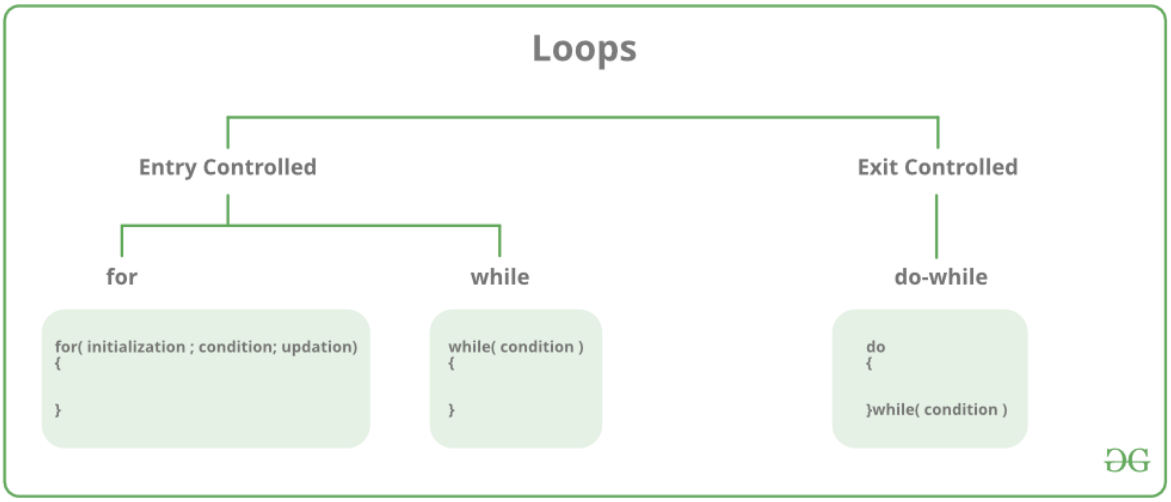
The example below uses the weekday number to calculate the weekday name:

Example

```
int day = 4;
switch (day) {
  case 1:
    cout << "Monday";
    break;
  case 2:
    cout << "Tuesday";
    break;
  case 3:
    cout << "Wednesday";
    break;
  case 4:
    cout << "Thursday";
    break;
  case 5:
    cout << "Friday";
    break;
  case 6:
    cout << "Saturday";
    break;
  case 7:
    cout << "Sunday";
    break;
}
// Outputs "Thursday" (day 4)
```

There are mainly two types of loops:

1. **Entry Controlled loops**: In this type of loop, the test condition is tested before entering the loop body. **For Loop** and **While Loop** is entry-controlled loops.
2. **Exit Controlled Loops**: In this type of loop the test condition is tested or evaluated at the end of the loop body. Therefore, the loop body will execute at least once, irrespective of whether the test condition is true or false. the do-while **loop** is exit controlled loop.



S.No.	Loop Type and Description
1.	**while loop** – First checks the condition, then executes the body.
2.	**for loop** – firstly initializes, then, condition check, execute body, update.
3.	**do-while** – firstly, execute the body then condition check

for Loop

A for loop is a repetition control structure that allows us to write a loop that is executed a specific number of times. The loop enables us to perform n number of steps together in one line.

Syntax:

```
for (initialization expr; test expr; update expr)
{
    // body of the loop
    // statements we want to execute
}
```

Example:

```
for(int i = 0; i < n; i++){
}
```

In for loop, a loop variable is used to control the loop. First, initialize this loop variable to some value, then check whether this variable is less than or greater than the counter value. If the statement is true, then the loop body is executed and the loop variable gets updated. Steps are repeated till the exit condition comes.

- **Initialization Expression**: In this expression, we have to initialize the loop counter to some value. for example: int i=1;
- **Test Expression**: In this expression, we have to test the condition. If the condition evaluates to true then we will execute the body of the loop and go to update expression otherwise we will exit from the for a loop. For example: i <= 10;
- **Update Expression**: After executing the loop body this expression increments/decrements the loop variable by some value. for example: i++;

Example:

```
// C program to illustrate for loop
#include <iostream>
Using namespace std;
int main()
{
    int i=0;

    for (i = 1; i <= 10; i++)
    {
        Cout<<”i=”<<i<<endl;
    }

    return 0;
   }
```

Output:

```
i=1
i=2
i=3
i=4
i=5
i=6
i=7
i=8
i=9
i=10
```

```
#include<iostream>
 using namespace std;
 int main()
 { int a,b,c;
 a=0;
 b=0;
 c=0;
 cout<<" Enter value for 'a'"<<endl;
 cin>>a;
 cout<<" Enter value for 'b'"<<endl;
 cin>>b;
 cout<<" Enter value for 'c'"<<endl;
 cin>>c;
 cout<<"_____"<<endl;
 cout<<"a= "<<a<<endl;
 cout<<""<<endl;
 cout<<"b= "<<b<<endl;
 cout<<""<<endl;
 cout<<"c= "<<c<<endl;
 cout<<""<<endl;
 if(a>b && a>c)
 {
     cout<<a<<"  is the Largest"<<endl;
 }
 else if (b>a && b>c)
  {
     cout<<b<<"  is the Largest"<<endl;
 }
 else if (c>a && c>b)
{
     cout<<c<<"  is the Largest"<<endl;
 }
 else
 {
     cout<<"All the values are equal..."<<endl;
     cout<<a<< " = "<<b<<" = "<<c<<endl;
 }
 }
```